CS-GY 9223 / CS-UY 3943 - Neuroinformatics, Spring 2026

Project 2 Phase 2: Kinematic Decoding from Neural Activity

Important Links & Deadline

Kaggle Competition:	Join Competition (Kaggle Link)
Team Sign-Up Sheet:	Sign Up Here (Google Sheets)
Deadline:	**March 29, 2026 at 11:59 PM EST (Sunday midnight)**
Code Submission:	Upload zipped code to **Brightspace**

Phase 2 of 2

This is **Phase 2** of a two-phase project.

- **Phase 1** focused on predicting masked neural activity (self-supervised reconstruction).
- **Phase 2** focuses on **decoding finger movements from neural signals** (supervised prediction of kinematics).
- Representations and insights learned in Phase 1 may carry over to Phase 2.

Note: You are allowed to use pre-built libraries (PyTorch, TensorFlow, scikit-learn, etc.) to implement your solution. You are free to use any pretrained models or additional datasets as well.

Submission Instructions:

- Submit a **final zipped code file** as your project submission on Brightspace.
- Use **Markdown cells** in notebook or neatly comment your code to include explanations wherever needed.
- You are allowed to use **LLMs**. If you do:
  - Include the **chat history** you had with the LLM along with your submission.
  - This helps us understand your **thinking style**.

- Participate in the **Kaggle competition** and submit your `submission.csv` file there.
- **Participation in Kaggle is mandatory, and your *private leaderboard* ranking will be used for final Project 2 Phase 2 scoring.**

Competition At A Glance:

Item	Details
Training Data	187 sessions of 96-channel intracortical neural recordings with kinematics
Test Data	125 sessions (SBP with $\sim$30% channels zeroed, kinematics withheld)
Evaluation	R^2 on $\sim$3.0M time bins $\times$ 2 position channels
Baseline	Per-session channel mean ($R^2 = 0.0$)

Quick Start:

1. Sign up your team on the Team Sign-Up Sheet
2. Join the Kaggle competition and download the dataset
3. Explore the 187 training sessions (SBP, kinematics, trial info)
4. Train a decoder to predict finger positions from neural activity
5. Generate predictions for all $\sim$3.0M time bins across 125 test sessions
6. Submit your `submission.csv` to Kaggle
7. Submit your code/notebook to Brightspace before the deadline

Contents

1 Problem Statement **4**
- 1.1 Background . . . 4
- 1.2 Challenge Goal . . . 4
- 1.3 What You Will Learn . . . 4

2 Data Description **4**
- 2.1 Dataset Overview . . . 4
- 2.2 Recording Setup . . . 5
- 2.3 Target Variables . . . 5
- 2.4 Decoding Approaches . . . 5
- 2.5 Channel Dropout . . . 5
- 2.6 Neural Drift . . . 6
- 2.7 The Task . . . 6
- 2.8 File Structure . . . 7
- 2.9 metadata.csv . . . 7
- 2.10 NumPy File Details . . . 7

3 Evaluation Metrics **8**
- 3.1 R^2 (Coefficient of Determination) . . . 8
- 3.2 Final Scoring . . . 8
- 3.3 Public/Private Leaderboard Split . . . 8

4 Baseline Results **9**
- 4.1 Key Takeaways . . . 10

5 Submission Instructions **10**
- 5.1 Required Output . . . 10
- 5.2 Important Requirements . . . 11
- 5.3 Creating a Submission . . . 11
- 5.4 Loading Data . . . 11
- 5.5 Report Submission Deadline . . . 12

6 Glossary **12**

1 Problem Statement

1.1 Background

Intracortical brain–computer interfaces (iBCIs) record neural activity from electrode arrays implanted in the brain. To restore motor control, these devices must **decode** neural signals into movement commands in real time. A fundamental challenge is **neural drift** — the statistical relationship between neural activity and behavior shifts over days, weeks, and months due to electrode migration, tissue remodeling, and neural plasticity. A decoder trained on one day's data can fail on recordings from weeks later.

1.2 Challenge Goal

Can we decode finger kinematics from neural activity across sessions spanning months to years of neural drift?

Task: Given long-term intracortical recordings spanning hundreds of sessions over multiple years from a non-human primate performing a finger-pressing task, **predict 2 finger position variables** (index and MRP finger positions) from 96-channel Spiking Band Power (SBP) — with ∼30% of channels randomly zeroed per session — at each time bin across 125 held-out test sessions.

1.3 What You Will Learn

- How neural activity encodes movement (neural tuning, population coding)
- Techniques for building neural decoders (Wiener filters, RNNs, transformers, etc.)
- How to handle distribution shift when training and test data are separated by months of neural drift

2 Data Description

2.1 Dataset Overview

The dataset consists of recordings from a 96-channel intracortical electrode array (two 64-channel Utah arrays) implanted in the motor cortex of a non-human primate. During each recording session, the subject performed a finger-movement task while neural activity and finger kinematics were captured simultaneously.

Table 1: Dataset Summary

Split	Sessions	Channels	Time Bins / Session	Sampling Rate
Train	187	96 (∼68 active)	∼21K–35K	50 Hz
Test	125	96 (∼68 active)	∼21K–35K	50 Hz

2.2 Recording Setup

- **96 channels** from two 64-channel Utah arrays in motor cortex
- **Spiking Band Power (SBP):** power in the spiking frequency band, extracted at 50 Hz (20 ms bins). Shape: `(N_bins, 96)`. ∼**30% of channels are randomly zeroed per session** (simulating electrode degradation).
- **Finger Kinematics:** 4 variables — index position, MRP position, index velocity, MRP velocity. Positions normalized to $[0, 1]$; velocities are derived from positions via discrete differentiation (`vel[t] = pos[t+1] - pos[t]`). Shape: `(N_bins, 4)`. **Only the 2 position channels are evaluated.**
- Each session: ∼375 trials, ∼21K–35K time bins (∼7–11 min of data)
- Recording span: ∼1,200+ days across 312 sessions

2.3 Target Variables

Table 2: Evaluated Position Channels

Column	Description	Range
`index_pos`	Index finger position (0 = extended, 1 = flexed)	$[0, 1]$
`mrp_pos`	Middle-ring-pinky position (0 = extended, 1 = flexed)	$[0, 1]$

Note: The kinematics files also contain velocity columns (`index_vel`, `mrp_vel`), which are simply the discrete time derivative of position: `vel[t] = pos[t+1] - pos[t]`. These are provided for convenience but are **not evaluated**. You may use velocity internally (e.g., as an auxiliary training target) if you wish.

2.4 Decoding Approaches

There are two natural strategies for predicting finger position:

1. **Direct position decoding:** Predict position directly from neural activity.
2. **Velocity-first decoding:** Decode velocity from neural activity, then integrate to recover position (since position $= \int \text{velocity}(t)\, dt$, or in discrete form, pos $=$ cumsum(vel)). Velocity is zero-anchored while position is relative to an arbitrary reference point, which can make it easier to decode. If you take this approach, calibrate your integrated output so the final position values are on the correct scale.

2.5 Channel Dropout

In each session, ∼30% of channels (28 out of 96) are randomly zeroed out (set to 0.0 for all time bins). The zeroed channels **differ per session**. You can identify active channels by checking which columns are all-zero:

```
sbp = np.load("train/D042˙sbp.npy")          # (N, 96)
active = ˜(sbp == 0).all(axis=0)             # True for active channels
print(f"Active: -active.sum()˝/96")           # ˜68/96
```

Your model must handle varying subsets of active channels across sessions.

2.6 Neural Drift

Mean SBP amplitude drops $\sim$42% over the recording span. The train/test split is chronological: training sessions span initial 60 % of data, test sessions span the rest. This ensures all test data is in the "future" relative to training, making neural drift the central challenge.

2.7 The Task

For each of the 125 test sessions, you are given SBP neural activity (with $\sim$30% of channels randomly zeroed) but **kinematics are withheld**. Your goal is to predict the 2 finger position variables at every time bin. You have 187 training sessions with paired SBP and kinematics to learn the neural-to-position mapping, then must apply your decoder to the test sessions.

2.8 File Structure

Listing 1: Dataset File Structure

```
        train/
              -session˙id˝˙sbp.npy              # (N, 96) ˜30% channels
zeroed
              -session˙id˝˙kinematics.npy       # (N, 4)  kinematics,
float32
              -session˙id˝˙trial˙info.npz       # trial start/end bins
        test/
              -session˙id˝˙sbp.npy              # (N, 96) ˜30% channels
zeroed
              -session˙id˝˙trial˙info.npz       # trial start/end bins
        metadata.csv          # session˙id, day, split, n˙bins, n˙trials
        test˙index.csv        # sample˙id, session˙id, time˙bin
        sample˙submission.csv # template for predictions
        metric.py             # official scoring function
```

2.9 metadata.csv

Table 3: Metadata Columns

Column	Description
session_id	Anonymous session identifier (D001, D002, ...) in random order
day	Relative recording day (day 0 = first session)
split	train or test
n_bins	Number of time bins in the session (∼21K–35K at 50 Hz)
n_trials	Number of trials in the session

2.10 NumPy File Details

Training sessions (train/):

- {sid}_sbp.npy — Float32 (N, 96). SBP values with ∼30% channels zeroed per session.
- {sid}_kinematics.npy — Float32 (N, 4). Columns: [index_pos, mrp_pos, index_vel, mrp_vel]. Positions in $[0, 1]$; velocities derived via vel[t] = pos[t+1] - pos[t]. **Only positions (first 2 columns) are evaluated.**
- {sid}_trial_info.npz — Contains start_bins, end_bins (arrays of trial boundary indices), and n_trials.

Test sessions (test/):

- {sid}_sbp.npy — Float32 (N, 96). SBP values with ∼30% channels zeroed. **Kinematics are NOT provided.**
- {sid}_trial_info.npz — Same format as training.

3 Evaluation Metrics

Submissions are evaluated by comparing your predicted positions to the hidden ground truth at all time bins across **125 test sessions** ($\sim$3.0M total time bins).

3.1 R^2 (Coefficient of Determination)

R^2 measures how well your predictions capture the variance in the true positions. For session s and position channel c:

$$R^2(s,c) \;=\; 1 \;-\; \frac{\displaystyle\sum_{i=1}^{n_s}\big(\hat{y}_i - y_i\big)^2}{\displaystyle\sum_{i=1}^{n_s}\big(y_i - \bar{y}\big)^2} \tag{1}$$

where n_s is the number of time bins in session s and $\bar{y}$ is the mean of the true values for channel c in session s.

3.2 Final Scoring

The final leaderboard score is the average R^2 across all (session, channel) groups:

$$\text{Final Score} = \frac{1}{250}\sum_{s=1}^{125}\sum_{c=1}^{2} R^2(s,c) \tag{2}$$

This ensures all 125 sessions and both position channels contribute equally.

Range: $(-\infty, 1]$ where 1 indicates perfect prediction (higher is better). On the Kaggle leaderboard, a **higher** R^2 corresponds to a **higher rank**.

Table 4: R^2 Interpretation

R^2	**Meaning**
$= 1.0$	Perfect prediction
> 0.0	Model captures meaningful position variance
$= 0.0$	Equivalent to predicting the per-session channel mean
< 0.0	Worse than the trivial baseline

3.3 Public/Private Leaderboard Split

Table 5: Leaderboard Configuration

Leaderboard	**Test Sessions**	**Count**
Public	64 sessions	51%
Private	61 sessions	49%

The public/private split uses **blockwise allocation** along the recording timeline — contiguous groups of test sessions are assigned to public or private. The final ranking uses the **private** portion.

Goal: Achieve R^2 well above 0.0, demonstrating that your decoder captures meaningful kinematic structure beyond the trivial mean baseline.

4 Baseline Results

Table 6: Baseline Model Results (R^2, with 30% channel dropout)

Model	R^2
Session mean (trivial)	0.000
Ridge Regression (Wiener filter, 10 lags)	-4.119
LSTM (2-layer, hidden=128)	0.199
GRU (2-layer, hidden=128)	0.213
Transformer + SSL pretraining	**0.448**

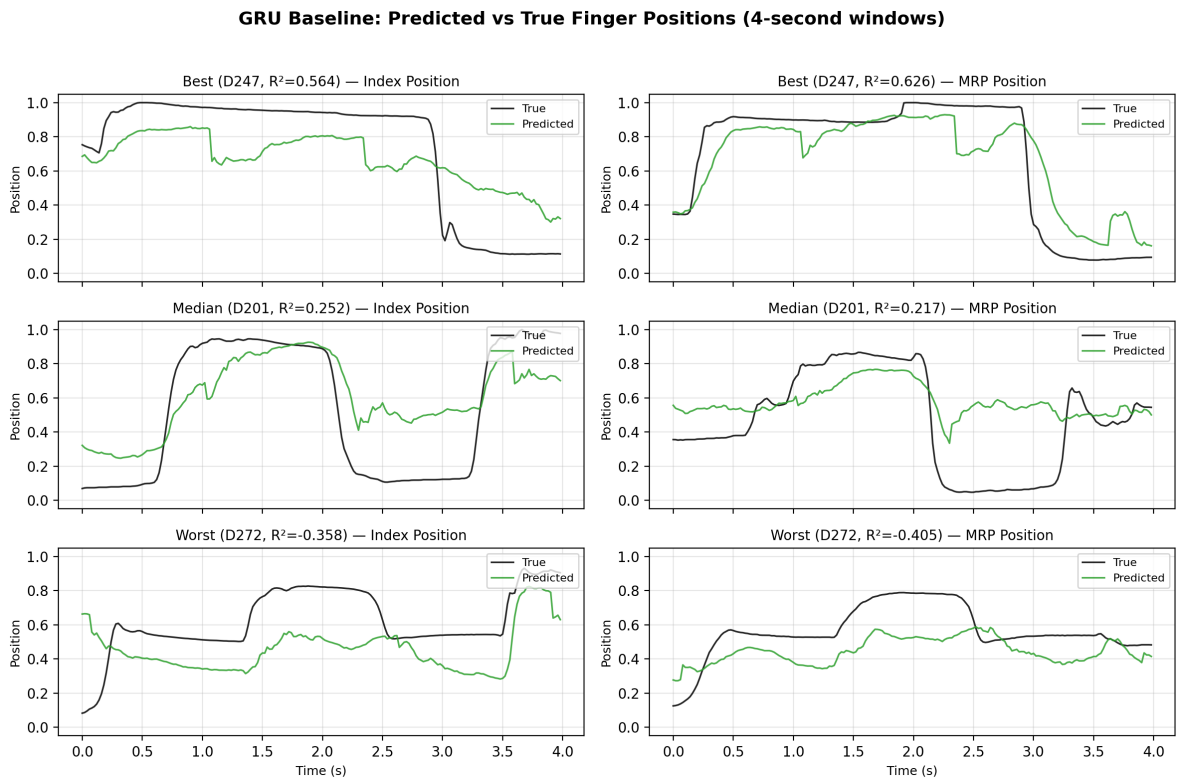

Figure 1: GRU baseline: predicted vs true finger positions on best, median, and worst test sessions (4-second windows). The GRU captures the general movement shape on good sessions but struggles with timing and amplitude on harder ones.

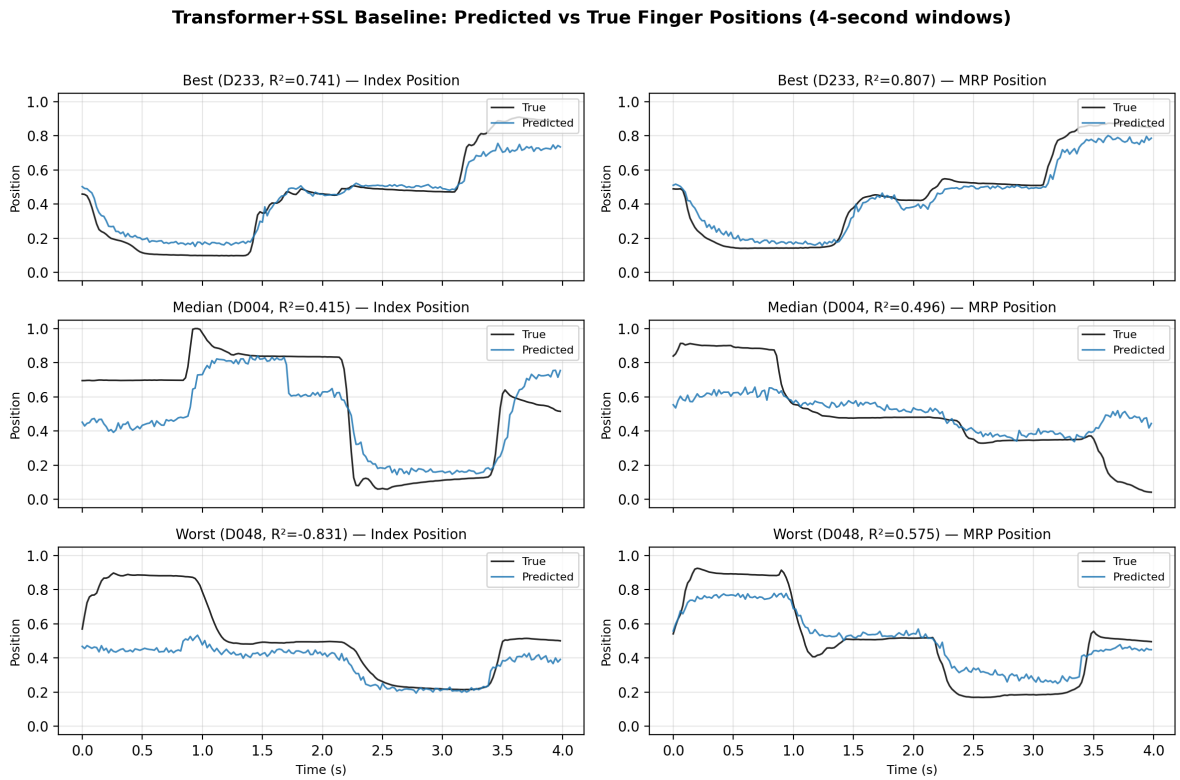

Figure 2: Transformer+SSL baseline: predicted vs true finger positions on best, median, and worst test sessions (4-second windows). Self-supervised pretraining helps the Transformer track finger movements more accurately, especially on best and median sessions.

4.1 Key Takeaways

- **The problem is solvable**: LSTM/GRU decoders achieve meaningful R^2 across sessions.
- **It is not trivially easy**: a naive Wiener filter (ridge regression) scores $R^2 \approx -4$ (much worse than predicting the mean).
- **Drift handling matters**: the key ingredient is per-session z-score normalization. Without it, cross-session models fail.
- **Channel dropout is challenging**: the $\sim$30% randomly zeroed channels require models that handle varying input subsets.
- **Room to improve**: the Transformer+SSL baseline achieves $R^2 = 0.45$. Self-supervised pretraining and domain adaptation can push higher.

5 Submission Instructions

5.1 Required Output

Submit a CSV file with five columns and $\sim$3.0M rows (one per test time bin):

Table 7: Submission File Format

Column	Description
`sample_id`	Integer index matching `sample_submission.csv`
`session_id`	Which test session (e.g., D009)
`time_bin`	Time bin index into the session's NumPy arrays
`index_pos`	Predicted index finger position (float)
`mrp_pos`	Predicted MRP finger position (float)

Example rows:

```
sample˙id,session˙id,time˙bin,index˙pos,mrp˙pos
0,D009,0,0.5,0.5
1,D009,1,0.5,0.5
...
```

5.2 Important Requirements

1. **Column names**: Exactly `sample_id,session_id,time_bin,index_pos,mrp_pos`
2. **Row count**: Must match `sample_submission.csv` (∼3.0M rows)
3. **Row ordering**: Your submission **must use the same `sample_id` ordering** as `sample_submission.c` Kaggle matches predictions to ground truth using `sample_id`. The easiest approach is to load `sample_submission.csv`, fill in your predictions, and save.
4. **No nulls**: All position values must be finite numbers (no NaN or Inf)
5. **Encoding**: UTF-8 CSV

5.3 Creating a Submission

Listing 2: Generating a Submission

```python
import pandas as pd

sub = pd.read˙csv("sample˙submission.csv")
# sub columns: sample˙id, session˙id, time˙bin,
#              index˙pos, mrp˙pos

# Replace kinematic columns with your predictions
# ... your model logic here ...

sub.to˙csv("submission.csv", index=False)
```

5.4 Loading Data

Listing 3: Loading Data Example

```
import numpy as np, pandas as pd

# Training session
sbp = np.load("train/D042˙sbp.npy")           # (N, 96)
kin = np.load("train/D042˙kinematics.npy")     # (N, 4)
trials = np.load("train/D042˙trial˙info.npz")  # start˙bins, end˙bins

# Test session (no kinematics!)
sbp˙test = np.load("test/D100˙sbp.npy")        # (N, 96)
trials˙t = np.load("test/D100˙trial˙info.npz")

# Metadata
meta = pd.read˙csv("metadata.csv")
train˙ids = meta[meta["split"]=="train"]["session˙id"].tolist()
test˙ids  = meta[meta["split"]=="test"]["session˙id"].tolist()
```

5.5 Report Submission Deadline

In addition to the Kaggle submission, you must submit a brief report describing the **methods and experiments** you performed.

Deadline: Sunday, March 29, 2026 at 11:59 PM EST.

The report may be provided either as:

- a PDF document, or
- documented Markdown / text cells within your notebook.

6 Glossary

Table 8: Terminology

Term	Definition
iBCI	Intracortical Brain–Computer Interface
SBP	Spiking Band Power (neural feature extracted from electrode signals)
Utah Array	Microelectrode array for intracortical recording
R^2	Coefficient of Determination (evaluation metric, higher is better)
Motor Cortex	Brain region controlling voluntary movement
Neural Drift	Change in neural signal statistics over time
MRP	Middle-Ring-Pinky (finger group)
Time Bin	20 ms interval (50 Hz sampling)
Wiener Filter	Linear decoder using time-lagged neural features

Good luck! We look forward to seeing innovative approaches to this challenging decoding problem.